

Tập 3 - Services & kube-proxy

Sự thật về ảo ảnh ClusterIP & Mổ xẻ bảng iptables

Phần 0 — Nền tảng K8s Networking · `#Service` `#ClusterIP` `#kube-proxy` `#iptables`



Lộ trình bài học: Lab First - Slide Second

Hôm nay chúng ta sẽ giải mã một trong những ảo ảnh lớn nhất của Kubernetes:

- **PHẦN 1: Thực hành đột nhập (Lab First)**

- Chứng kiến cú lừa kinh điển: IP ảo `ping` thì chết nhưng `curl` lại chạy.
- Chui xuống mức OS để lùng sục dấu vết `iptables` và đo tỷ lệ load balance ngẫu nhiên.
- Soi "nhật ký dịch chuyển" bằng `conntrack`.

- **PHẦN 2: Đúc kết bản chất (Slide Second)**

- Phanh phui bản chất: ClusterIP có card mạng và địa chỉ MAC hay không?
- Sơ đồ hóa chuỗi điệp vụ `iptables` do `kube-proxy` lập trình.
- Cách NodePort mở cổng trên toàn cụm để dẫn đường cho traffic.

PHẦN 1: Bắt đầu làm Lab ngay!

Chúng ta sẽ thực hiện các Thí nghiệm cân não trong file `lab-guide.md`:

1. **Thí nghiệm 1:** Tạo Service ClusterIP cho Nginx. Chứng minh cú sốc `ping ClusterIP` bị timeout nhưng `curl` lại trả về HTML vèo vèo.
2. **Thí nghiệm 2:** Lên `worker1`, gõ grep iptables để lần theo các chuỗi `KUBE-SERVICES` -> `KUBE-SVC-xxx` -> `KUBE-SEP-xxx` để thấy xác suất rẽ nhánh thống kê.
3. **Thí nghiệm 3:** Kiểm tra cache của module NAT thông qua lệnh `conntrack`.
4. **Thí nghiệm 4:** Chuyển sang NodePort và dùng IP của bất kỳ Node nào để truy cập ứng dụng.

👉 Hãy tạm dừng video, mở terminal và thực hiện lab theo tệp `lab-guide.md` nhé!

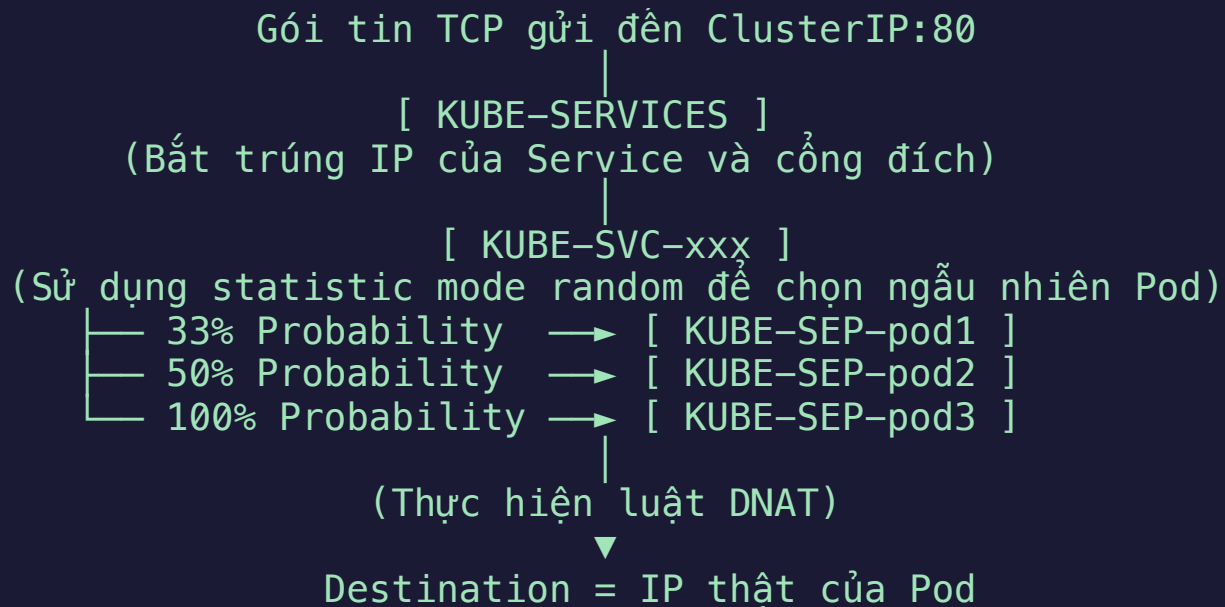
PHẦN 2: Giải mã hiện tượng Ping Timeout vs Curl Success

Tại sao ClusterIP lại có hành vi kỳ lạ như vậy?

- 🖱️ **Câu trả lời: ClusterIP hoàn toàn không có thực!** Nó không được gán vào bất kỳ card mạng (NIC) vật lý hay ảo nào cả, và cũng không có địa chỉ MAC. Nó chỉ là một **ảo ảnh**!
- Khi bạn `ping <ClusterIP>` (giao thức ICMP), gói tin gửi ra card mạng sẽ không có ai phản hồi vì IP đó không tồn tại trên thực tế -> Dẫn đến Timeout.
- Khi bạn `curl <ClusterIP>:80` (giao thức TCP), `iptables` trên chính Node đó lập tức bắt gói tin (match rule), bẻ lái địa chỉ đích (DNAT) thành IP thật của Pod trước khi gói tin kịp rời khỏi Node!

Sơ đồ luồng đi của gói tin qua Iptables

Khi bạn tạo một Service, `kube-proxy` sẽ chạy ngầm và cấu hình bảng `iptables -t nat` trên **MỌI NODE** theo sơ đồ chuỗi sau:



Conntrack & NodePort: Cách K8s dẫn đường

1. Conntrack (Connection Tracking)

Khi iptables thực hiện DNAT (đổi IP ảo -> IP thật), nó cần ghi nhớ "nhật ký" ở module `conntrack` của Linux Kernel. Khi Pod phản hồi ngược lại, `conntrack` tự động dịch ngược (SNAT) thành IP ảo để Client không nhận ra sự thay đổi.

2. NodePort: Mở cổng trên toàn cụm

Khi đổi Service sang NodePort (ví dụ: `31234`), `kube-proxy` sẽ binding port đó trên **tất cả các card mạng của tất cả các node**. Iptables sẽ bắt traffic đập vào port này trên bất kỳ node nào và tự động chuyển hướng nó về đúng node chứa Pod.

Key Takeaways — Bài học cốt lõi

- **ClusterIP là ảo ảnh:** Toàn bộ cơ chế Service thực chất là một hệ thống phân phối luật **NAT (Network Address Translation)** khổng lồ do `kube-proxy` lập trình tự động trên nền tảng `iptables` của Linux Kernel.
- **Load Balancing cấp hạt nhân:** K8s sử dụng tính năng xác suất thống kê có sẵn của Linux (`statistic mode random`) để chia tải, rất nhẹ và hiệu năng cao.
- **Hạn chế của iptables:** Khi cụm có hàng ngàn Service, số lượng dòng iptables tăng lên tới hàng chục vạn dòng. Linux phải duyệt tuần tự $O(N)$ dẫn đến giảm hiệu năng -> Đây là lý do IPVS và eBPF ra đời!

Tập tiếp theo: Làm thế nào để các Pod gọi nhau qua Tên thay vì IP ảo? Khám phá CoreDNS và sắc thuế DNS "ndots:5" cực kỳ lãng phí!